DISTRIBUTED AI RESEARCHER

Sumang's Signature Edition - The Comprehensive Masterpiece Documentation

---

Part 1: The Genesis and Architectural Decisions

The Distributed AI Researcher was conceived to solve a specific problem: standard AI models are lazy. If you ask ChatGPT to research a massive topic, it will write a quick 5-paragraph essay based on its pre-trained data and stop.

Our goal was to build **Swarm Intelligence**: an AI application that behaves like a team of human researchers. When given a complex prompt, it should break the prompt down, browse the live internet, read multiple articles simultaneously, and synthesize a massive master report.

1.1 Why Separate Backend (Python) and Frontend (Next.js)?

We needed extreme parallel processing capabilities.

- Node.js (JavaScript) is single-threaded. If we tried to run 5 AI web searches at the exact same time in Node.js, it would queue them up and run them one by one, making the user wait forever.
- **Python's `ThreadPoolExecutor`** allows us to spin up actual, physical system threads. We can deploy 5 worker agents simultaneously. They all search the web at the exact same time, analyze the data at the exact same time, and finish in 1/5th of the time.

1.2 The Database Dilemma: Supabase vs Pinecone

For VISION AI (our other project), we used Pinecone because we needed vector math to retrieve semantic memories.
But for Distributed AI Researcher, we used **Supabase (PostgreSQL)**.
Why? Because a research report is a structured document. We needed a reliable relational database to store the `query` (the title), the `report` (the massive markdown text), and the `created_at` timestamp. Supabase acts as our immortal cloud storage. Even if your local PC explodes, your research is saved forever in the cloud and instantly fetched by the frontend `History` sidebar.

1.3 The Queue Architecture: Redis

We integrated **Upstash Redis** as our message broker.
Why didn't we just run the research directly? Because web research takes 30 to 60 seconds. If a user clicks "Search" on the frontend, standard HTTP connections time out after 30 seconds, causing the website to crash with a 504 Gateway Error.
By using Redis, when the user clicks "Search", the backend instantly throws the job into a Redis Queue and replies "Job received!". The background worker slowly processes the queue, and the frontend simply polls the server every 2 seconds asking "Is it done yet?" This guarantees the website never freezes or crashes, no matter how long the AI takes to research.

Part 2: File-by-File Technical Breakdown

🗁 `backend/main.py` (The Orchestrator)

This is the FastAPI gateway.

- It exposes the `/api/research` endpoint.
- It handles the HTTP requests from the Next.js frontend, safely queuing jobs into Redis.

🗁 `backend/worker.py` (The Workhorse)

This file is a daemon (a background process that runs in an infinite loop).

- **The Infinite Loop:** It uses `redis_client.brpop("research_jobs")` to wait for a job.
- **The Execution:** Once a job appears, it calls `execute_research(query)`.
- **The Archival Process:** It saves the final report to Redis (for the frontend to grab) and injects it into Supabase PostgreSQL for permanent storage.
- *The Bug Fix:* Originally, `worker.py` imported `supabase` and `redis`, but they were missing from `requirements.txt`. This caused an immediate boot crash. We permanently fixed this by appending all dependencies to `requirements.txt`.

🗁 `backend/ai_swarm.py` (The Architect)

This is the true brain of the operation.

1. **The Blueprint:** It sends the user's prompt to LLaMA 3.3 and says, "Act as an Architect. Break this topic down into 3-5 sub-queries."
2. **The Swarm:** It takes those sub-queries and feeds them into a `ThreadPoolExecutor`.
3. **The Web Research:** Every thread contacts the Tavily Search API, which actually browses Google in real-time, reads the websites, and returns the text.
4. **The Synthesis:** The Architect takes all 5 mini-reports and asks LLaMA 3.3 to weave them into a flawless Markdown document.

🗁 `frontend/app/page.tsx` (The Dragonic Cyber-Grid UI)

The visual layer is designed to inspire awe.

The 3D Grid Mathematics:

```
<div className="absolute inset-0 origin-bottom [transform:rotateX(60deg)_translateZ(-200px)] opacity-90">
```

Why we did this: We didn't want a flat image background. By utilizing CSS `rotateX(60deg)`, we took a flat 2D grid and tilted it backward in 3D space. By animating a `linear-gradient` to slide continuously, it creates an optical illusion of moving forward through an infinite digital floor.

The Bug We Fixed: Similar to VISION AI, we originally painted a solid `bg-[#050505]` color over the main container, which accidentally hid the 3D grid! We removed this wall, making it `bg-transparent`, and cranked the red opacity to 90% so the Dragonic Vibe is massively prominent.

The Polling Mechanism:

```
const interval = setInterval(async () => {
    const res = await fetch(`${API_BASE}/api/job/${jobId}`);
    // Check if job is finished
}, 2000);
```

Why we did this: Since HTTP can't wait 60 seconds, React uses `setInterval` to "poll" the backend. Every 2 seconds, the frontend pings the server silently in the background. Once the backend says "Finished!", React clears the interval and renders the report.

🗁 Configuration Files (`package.json`, `tsconfig.json`, etc.)

These files must remain in the root directory. Next.js uses them to understand how to build the React application, process the Tailwind CSS, and deploy to Vercel. Moving them into a `docs` or `backend` folder would completely sever Vercel's ability to host the site.

---

3. Conclusion

The Distributed AI Researcher represents elite architectural engineering. It utilizes Redis for asynchronous queue management, Supabase for persistent cloud storage, Multi-threading for concurrent AI execution, and 3D CSS rendering for an unforgettable user experience.